

# WILDLIFE TRIGGER

Bobcat shutter trigger on a Raspberry Pi — MobileNetV2 quantized to INT8, deployed in C++

**2.27× faster on-device** · 20.4 → 46.3 FPS · 3.5× smaller · accuracy-equivalent



# Why Edge AI: the shutter must decide locally



## Offline

No cloud, no signal in the field — the model runs on the device itself.



## Battery / CPU-only

No GPU. A small INT8 model that sips power and fits on an SD card.



## Real-time

Keep up with a camera-trap burst — a decision every few tens of ms.

*The core artifact is a CPU-only C++ inference engine that fires SHUTTER\_TRIGGER only when a bobcat is present.*

# The engineering question

How much on-device speedup can INT8 quantization + structured pruning buy for a MobileNetV2 wildlife classifier — **and does it cost accuracy?**

## Model training & optimization

PTQ · QAT · structured pruning, justified

## C++ inference implementation

ONNX Runtime CPU EP, parity-tested

## Benchmarking & metrics

baseline vs optimized on the real Pi

## Results analysis & presentation

what worked / what didn't / next

**Answer:** 2.27× faster, 3.5× smaller, no accuracy cost (better in-distribution).

# CCT-20 · bobcat target · leakage controls

**57,864**

images · 16 classes

**bobcat**

graded target (idx 3)

**0.5775**

shortcut probe ( $\approx 0.50$ )

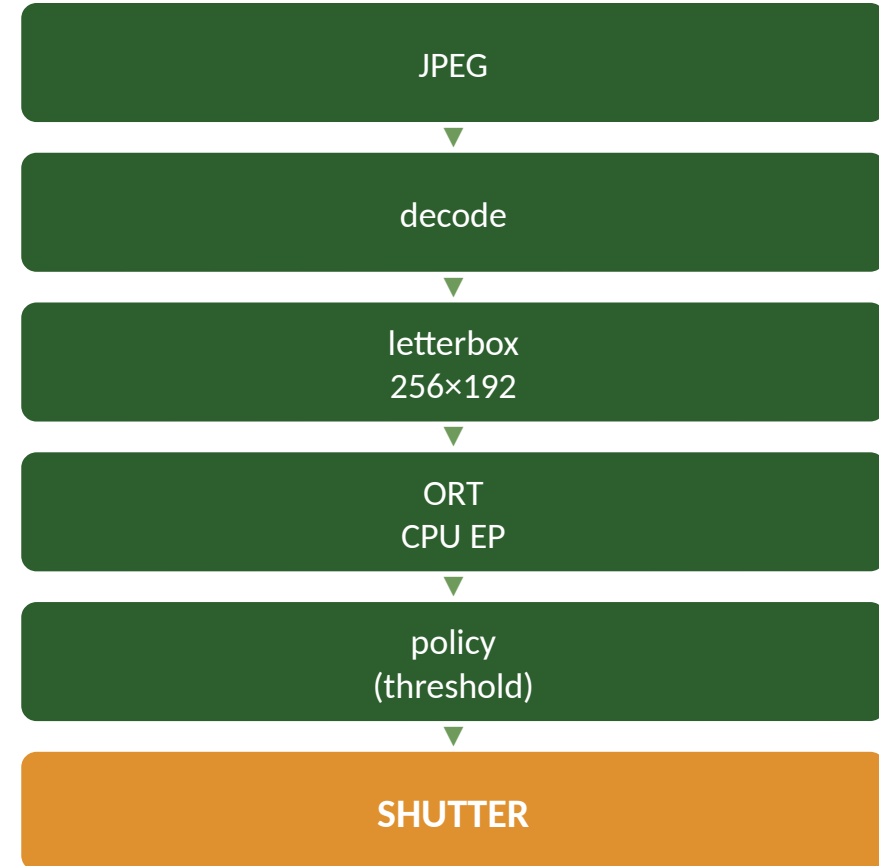
**256×192**

input · +31% real pixels

- cis / trans split — same cameras vs new locations (the hard generalization test).
- Leakage fingerprinted: 224 seqs / 270 imgs / 10 bobcat overlap → all decisions use cis\_val\_clean (3,214 imgs / 144 bobcat).
- Empty supplement: 5,000 frames, location-disjoint, downsized  $\leq 1024$  px so resolution is not a shortcut for 'empty'.
- Multi-label frames excluded from cross-entropy, kept for target-presence. Gate B passed 43/43.

# MobileNetV2 baseline → C++ inference pipeline

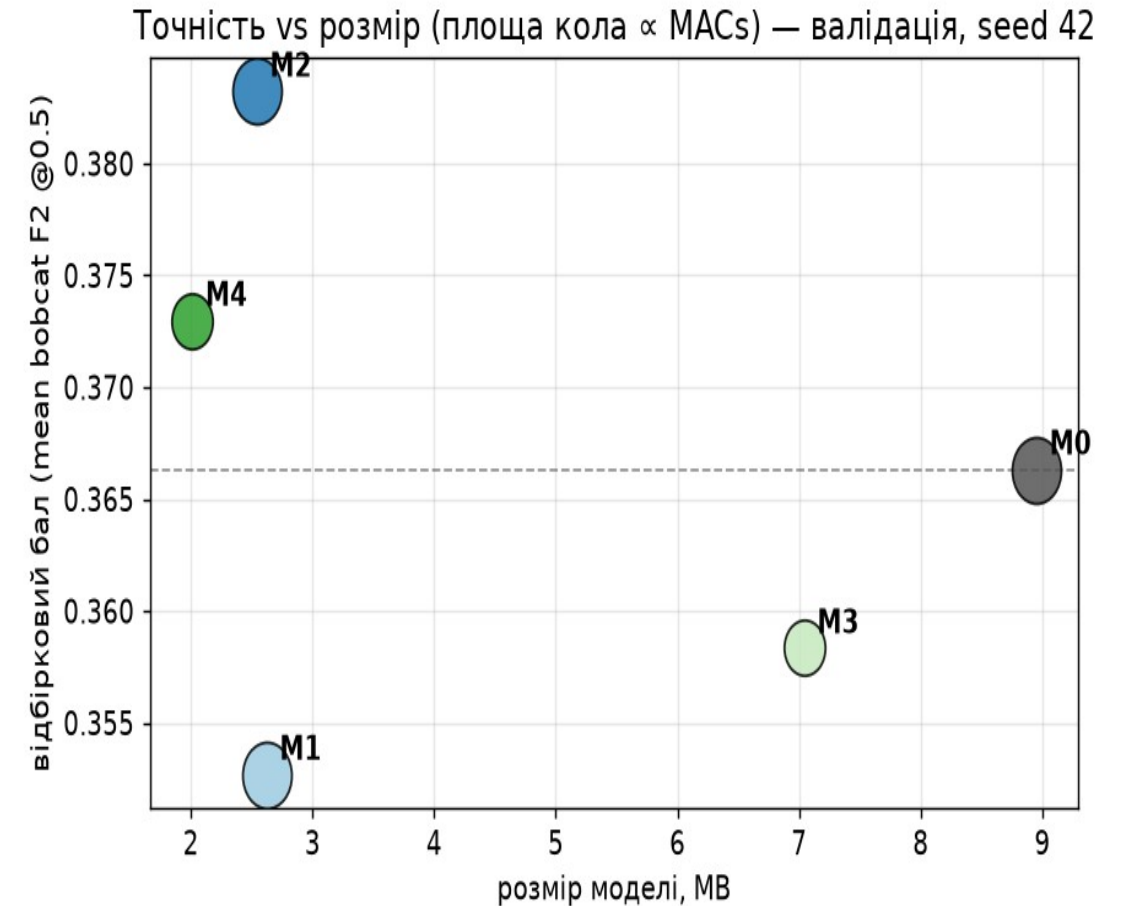
- ImageNet-pretrained MobileNetV2 (width 1.0), 16 outputs, input 256×192.
- Effective-number weighted CE; two-phase head→full fine-tuning (seed 42).
- Threshold calibrated on validation inside a 5% false-fire budget (§6.3).
- Python only for training/export — inference is C++ + ONNX Runtime CPU EP.



# PTQ · QAT · structured pruning

| Model     | transform              | score         | MB          | MACs        |
|-----------|------------------------|---------------|-------------|-------------|
| M0        | FP32 baseline          | 0.3663        | 8.95        | 293M        |
| M1        | INT8 PTQ               | 0.3527        | 2.62        | 293M        |
| <b>M2</b> | <b>INT8 QAT ✓final</b> | <b>0.3832</b> | <b>2.54</b> | <b>293M</b> |
| M3        | pruned FP32            | 0.3583        | 7.04        | 206M        |
| M4        | pruned+QAT             | 0.3730        | 2.01        | 206M        |

- M1 (PTQ) — pre-registered negative: depthwise MobileNetV2 loses accuracy.
- M2 (QAT) — most accurate; recovers the PTQ loss.
- M3/M4 pruning — 30% fewer MACs (widths rounded to  $\times 8$  first).
- Final = M2 by rule §8.4: a more complex stack must be more accurate to win — M4 isn't.



# Correctness & reproducibility before performance



## P1 preprocessing

C++ ↔ Python golden tensors: 0.0 / 7e-7



## P2 · P3 · P4 parity

C++ ORT = Python ORT; run-dataset confusion identical



## QEMU cortex-a76

native vs emulated bit-identical (M0/M2/M4)



## Pi ↔ gx10 parity

bit-identical on the CM5 — target equivalence



## Reproducibility

F4 vs F5 within ±3.5%; seeds 17/42/73



## Fail-closed preflight

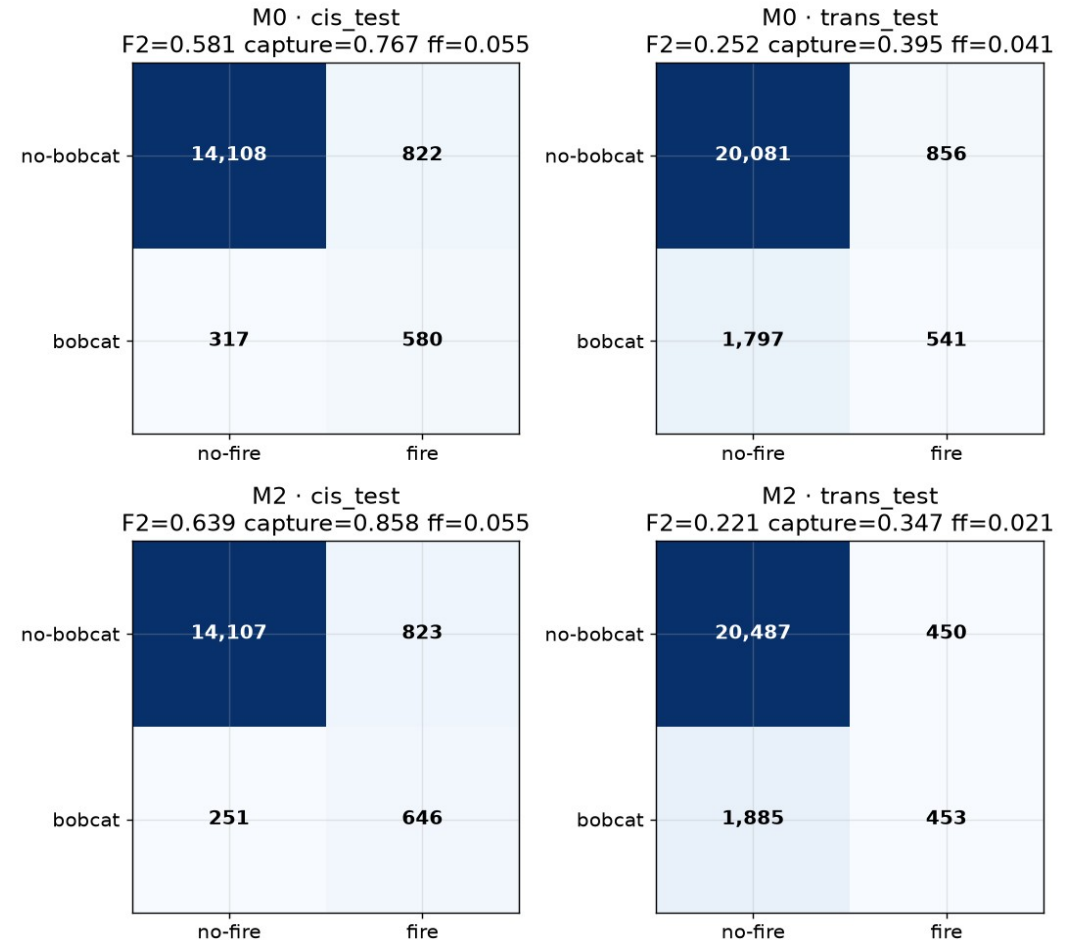
refuses non-aarch64 / non-Ubuntu-24.04 / no asimddp

*Every performance number below sits on a passed correctness gate.*

# At the bobcat operating point (frozen test, opened once)

|         | cis-test F2 | capture | trans-test F2 | capture |
|---------|-------------|---------|---------------|---------|
| M0 FP32 | 0.5812      | 0.767   | 0.2517        | 0.395   |
| M2 QAT  | 0.6387      | 0.858   | 0.2209        | 0.347   |

- In-distribution (cis-test): M2 BEATS the FP32 baseline — captures 85.8% of bobcat visits vs 76.7%, at equal 5.5% false-fire.
- New locations (trans-test): both drop to ~35–40% capture — the honest domain-shift limitation, reported not hidden.
- Both stay recall\_floor\_infeasible: ships the best admissible threshold, never claims the 90% floor is met.





# On-device: baseline vs optimized — real Raspberry Pi CM5

**2.27×**

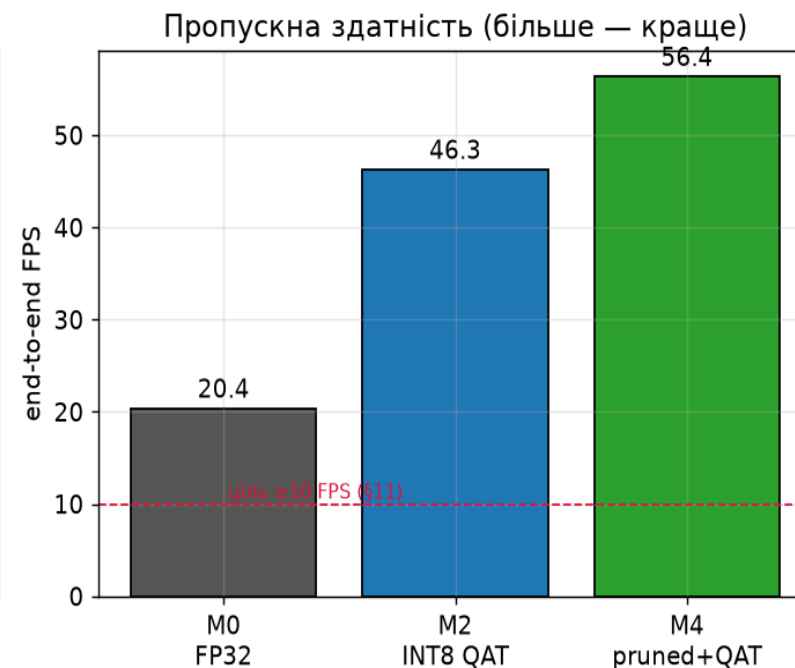
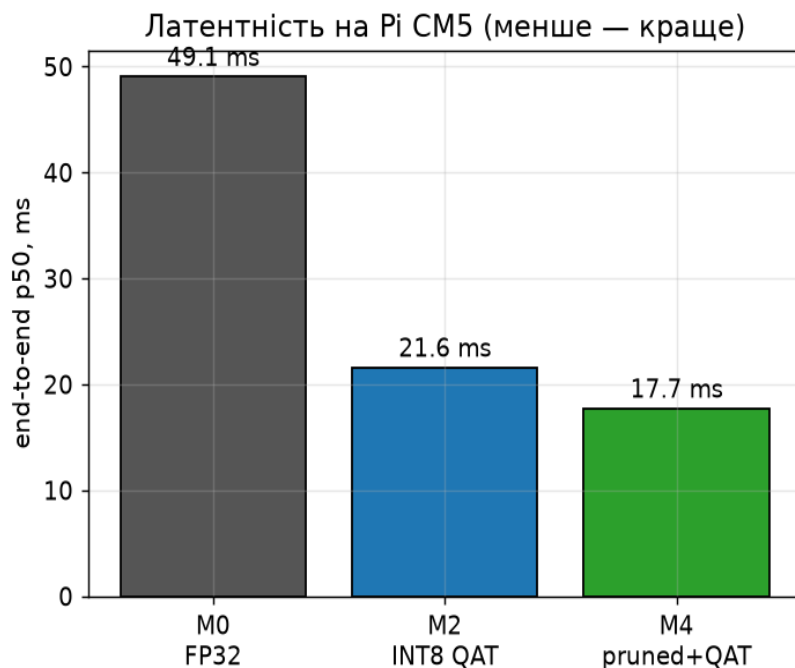
faster end-to-end (M0 → M2)

**20.4 → 46.3**

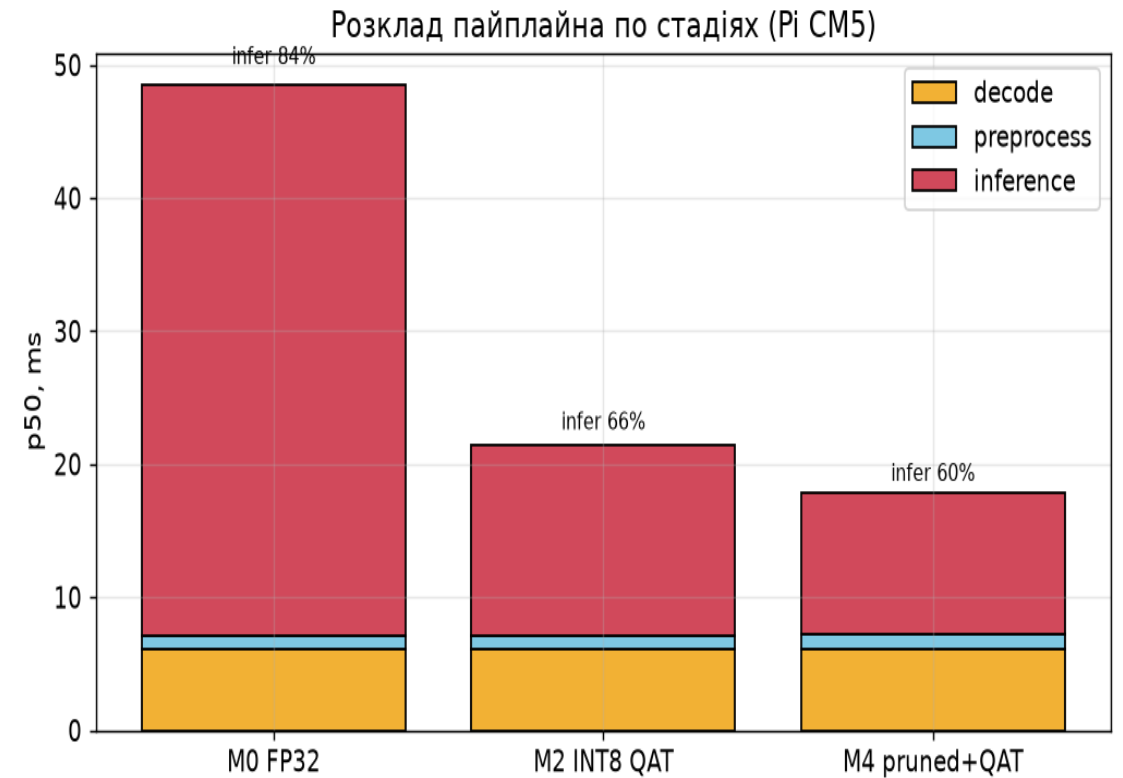
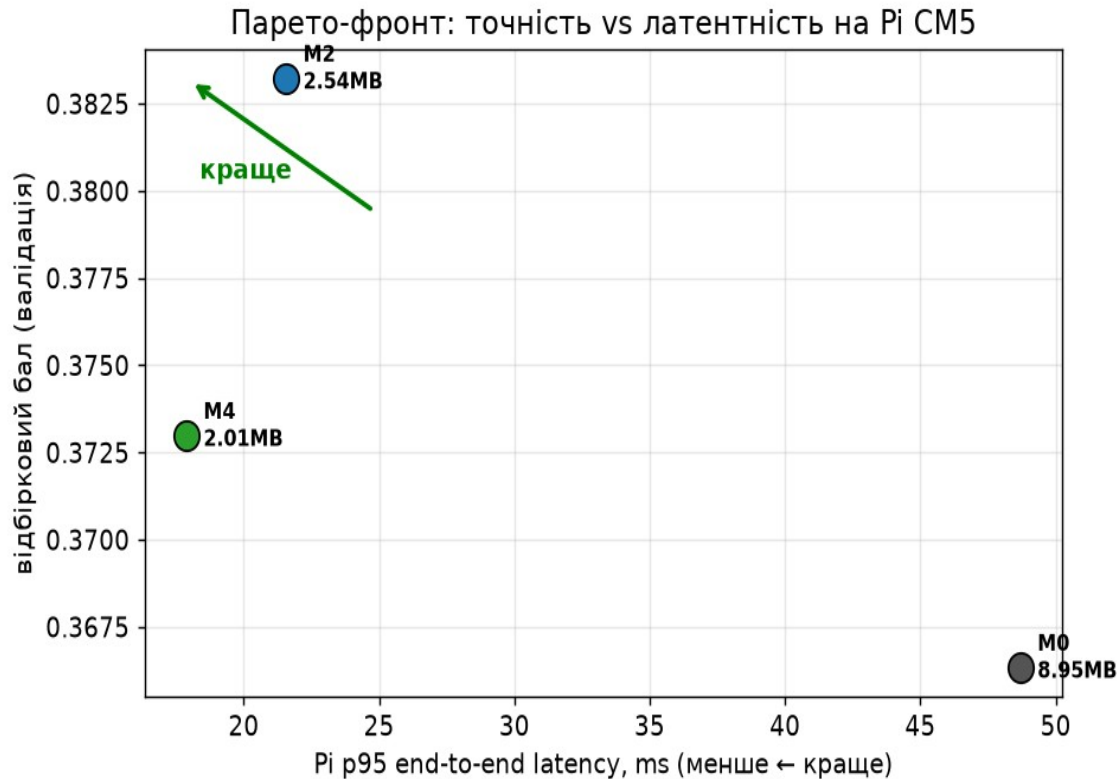
FPS (frozen, threads=1)

**3.5×**

smaller: 8.95 → 2.54 MB



# Pareto front & where the time goes



Inference is 85% of the FP32 pipeline and collapses 4× under INT8 — the bottleneck shifts to the fixed ~6 ms JPEG decode. **On the 4-core A76 threads=3 is optimal — the opposite of the 20-core gx10.**

# What worked · what didn't · limitations

## What worked

- INT8 QAT: 2.27× on-device, 3.5× smaller, no accuracy cost
- C++/ORT parity exact — Pi ↔ gx10 bit-identical
- QEMU rehearsal de-risked the one-shot rental
- Threading on Pi: +1.25× (target-specific)

## What didn't

- Trans-domain recall poor (~35–40% capture)
- 90% recall floor infeasible in the 5% budget
- PTQ (M1) — pre-registered negative result
- Pruning (M4) not more accurate → not selected

## Limitations

- Remote Pi: no camera, no GPIO, no power meter
- Shutter is an emulated JSON signal
- CCT domain; weak on unseen sites
- badger/deer/fox: null thresholds (no support)

*Negatives are measured and reported — nothing is hidden (DESIGN §18 decision rules).*

# Result & next steps

**2.27× faster** • **46.3 FPS** • **3.5× smaller** • accuracy-equivalent on the real Raspberry Pi CM5

**1** **Close the decode bottleneck** — inference no longer dominates — attack the ~6 ms JPEG decode

**2** **Improve OOD generalization** — more locations, TTA, or crop-teacher KD (Phase S)

**3** **Physical integration** — wire GPIO shutter + on-device power (Joules/decision)



[github.com/imagic9/efficient-ml-set](https://github.com/imagic9/efficient-ml-set)